COMP3320/COMP6464 Week 1, Lecture 3

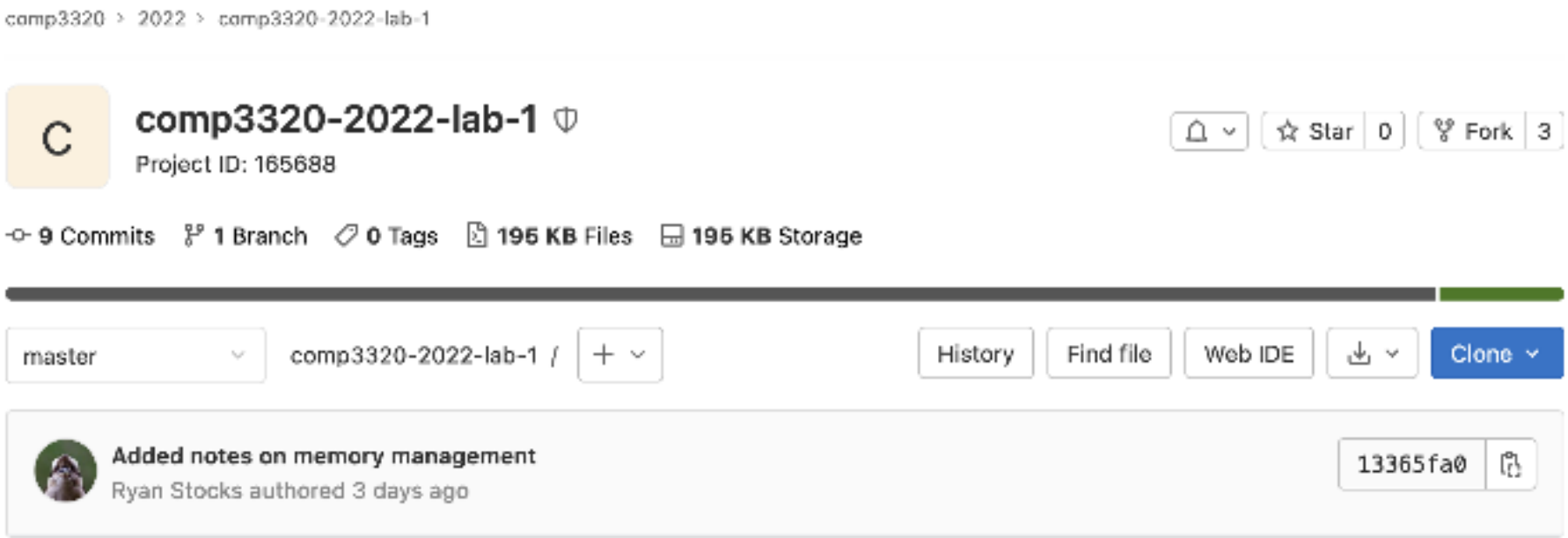
Rhys Hawkins

Semester 2, 2024

Labs

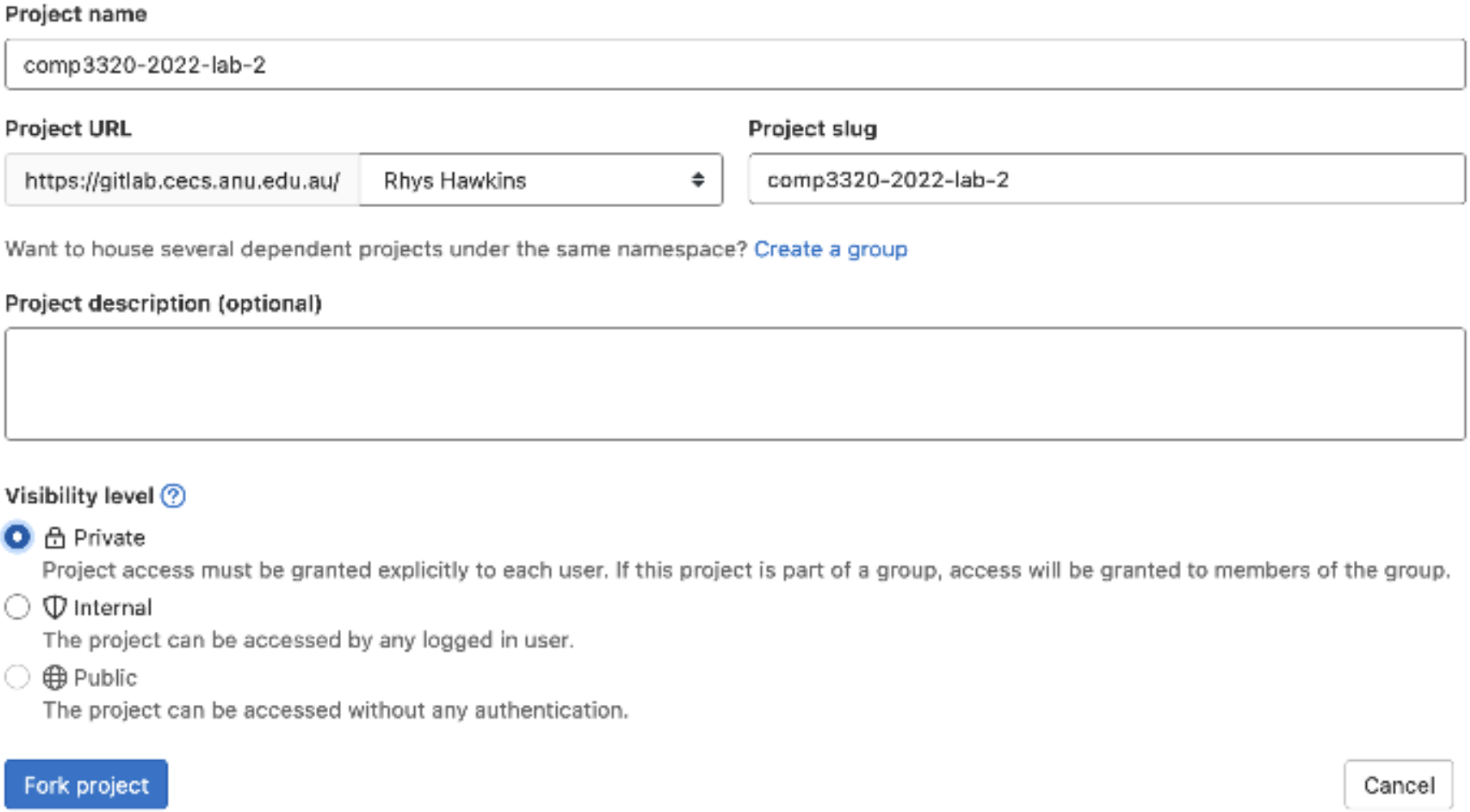
- Starting next week
- Sneak Peak
- `https://gitlab.cecs.anu.edu.au/comp3320/2024/comp3320-2024-lab-1`

Labs



- Select Fork (top right)

Labs



- Select your own user name
- Private

Labs

- On Gadi (or your own computer)
- `> git clone git@gitlab.cecs.anu.edu.au:<your anu id>/comp3320-2024-lab-1.git`

Labs

README.md

COMP3320/6464 2022: Laboratory 1

Getting Started and Performance Benchmarking

The aim of this lab is to get you up and running on the NCI facility, then to give you some experience of timing code, performance measurement and benchmarking

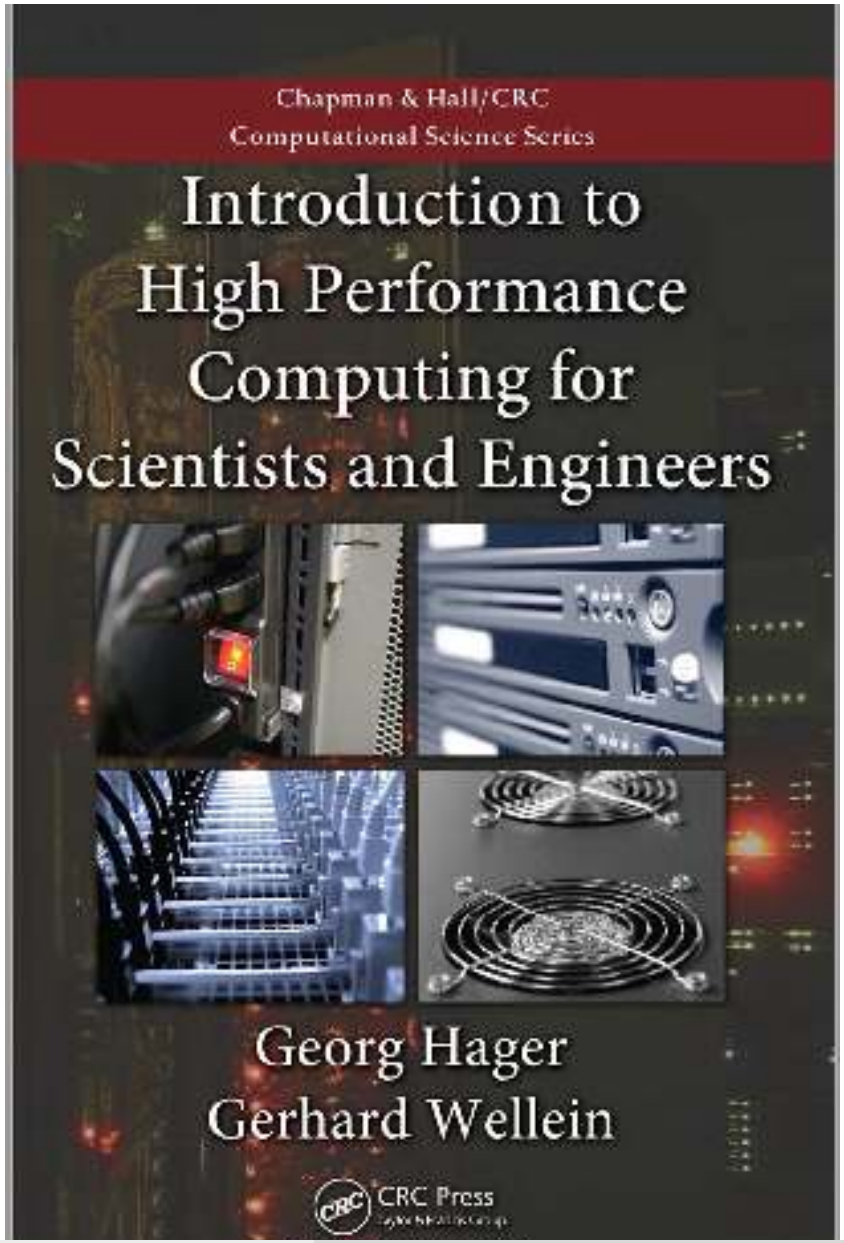
Getting Started on the NCI Gadi System

Objective - to get you up and running on Gadi

- Detailed instructions in the README.md file including basic GADI instructions, GIT instructions and C-primer
- `https://gitlab.cecs.anu.edu.au/comp3320/2024/comp3320-2024-lab-1`

Textbook

Chapter 1 except 1.6 (Vector Processors)



Questions and Answers

- Are there any questions about the course, labs, etc

Profiling Internals

Approaches

- Instrumentation - adding performance measurement calls at various parts of code or binary, eg. gprof, various tools for profiling java code
- Sampling - statistically sample performance data at regular intervals during program execution,
- Simulation - run binary on a virtual machine which simulates actual hardware.

Challenges

- Analyzing performance becomes increasingly difficult from User Application to Operating system to Hardware.

Advantages and Disadvanges of Different Profiling Approaches

- Instrumentation
  - Advantages: Can precisely measure a given aspect of performance
  - Disadvantages: It is not very efficient especially when we want to measure many things at once. May have side effects.
- Sampling
  - Advantages: The most flexible approach for measuring many aspects of performance at once. Very efficient. Few side effects.
  - Disadvanges: Depending on sampling rate, may not be entirely precise.
- Simulation
  - Advantages: Can be very precise. May be the best method for understanding what goes on in hardware.
  - Disadvantages: Very difficult to implement accurately. Very inefficient.
  - Intel CPU internal operation hidden.

Subroutine Profiling: gprof

- Normal compilation: `>gcc -o nbody nbody.c -lm`
- GProf compilation: `>gcc -o nbody nbody.c -lm -pg`
- Run `nbody` normally
- `gprof ./nbody` outputs

```
  %   cumulative   self              self     total
 time   seconds   seconds    calls  ms/call  ms/call  name
 99.04     14.45    14.45     1000    14.45    14.45  compute_forces
  0.75     14.56     0.11                             _init
  0.21     14.59     0.03     1000     0.03     0.03  update_velocties
  0.00     14.59     0.00     7000     0.00     0.00  random_double
  0.00     14.59     0.00     1000     0.00     0.00  update_positions
...
```

Line Counting: gcov

- Normal compilation: `>gcc -o nbody nbody.c -lm`
- gcov compilation: `>gcc -o nbody nbody.c -lm -coverage`
- Run `nbody` normally
- `>gcov nbody.c`
- `>less nbody.c.cov`
- Counts number of time line was executed, not the time required!

```
...
  1001000:   46:  for (int i = 0; i < N; i ++) {
500500000:   47:    for (int j = i + 1; j < N; j ++) {
        -:   48:
499500000:   49:      double dx = bodies[j].x - bodies[i].x;
499500000:   50:      double dy = bodies[j].y - bodies[i].y;
499500000:   51:      double dz = bodies[j].z - bodies[i].z;
        -:   52:
...
```

Intel VTune Performance Analyzer

A sampling based performance analyzer tool for Intel processors Features

- Command Line and GUI Interface
- Call graph profiling, similar to gprof
- Basic block profiling down to individual assembly instructions
- Hardware performance counters
- Analysis of threaded programs
- System wide profiling, including the operating system
- Must compile application using ”-g” option
- Too much to cover

Command Line VTune On Gadi

- `ssh -Y xxx444@gadi.nci.org.au`
- `module avail`
- `module load intel-vtune`

Basic Command Line VTune on Gadi

- `vtune -collect performance-snapshot ./nbody <args>`
- Will output statistics to a directory `rxxxps` with `xxx` a number
- `vtune -report summary -r r000ps`
- Outputs a basic report with suggestions, e.g.

```
Recommendations:
    Increase execution time:
     | Application execution time is too short. Metrics data may be unreliable.
     | Consider increasing your application execution time.
    Hotspots: Start with Hotspots analysis to understand the efficiency of your algo
     | Use Hotspots analysis to identify the most time consuming functions.
     | Drill down to see the time spent on every line of code.
```

Basic Command Line VTune on Gadi

- `vtune -collect hotspots ./nbody <args>`
- Will output statistics to a directory `rxxxhs` with `xxx` a number
- `vtune -report hotspots -r r000hs`
- Outputs a gprof like output with time usage by function, e.g.

```
---------------  --------  ----------------------
compute_forces     10.520s                10.520s
__libm_sqrt_ex      2.830s                 2.830s
printf              0.010s                 0.010s
update_positions    0.010s                 0.010s
```

Basic Command Line VTune on Gadi

- More detailed information available per line
- `vtune -report hotspots -r r000hs -group-by source-line`
- Outputs a hotspots on a per line basis, e.g.

```
--------------------  -----------  --------
[Unknown source file]  [Unknown]      2.840s
nbody.c                56             2.810s
nbody.c                54             0.910s
nbody.c                60             0.680s
nbody.c                58             0.560s
nbody.c                69             0.550s
nbody.c                57             0.540s
```

Running VTune GUI on Gadi

- Enable X11 Forwarding
- `ssh -Y xxx444@gadi.nci.org.au`
- `model load intel-vtune`
- `vtune-gui`
- Will be slow (especially for off campus students)
- Another option is to download Intel Compilers (students can do so for free)

Intel Advisor

- VTune is a performance measurement tool
- Advisor is that plus a tool to highlight parts of your code that could be improved
- `module load intel-advisor`

Intel Advisor : Basic Command Line Usage

- `advisor --collect=survey -- ./nbody`
- `advisor --report=survey`
- Example output (first few columns)
- Notice that advisor looks a time consuming loops

```
2   loop in compute_forces at nbody.c:47        1.390s      1.050s   Scalar
1   loop in main at nbody.c:156                 1.390s          0s   Scalar
3   loop in compute_forces at nbody.c:46        1.390s          0s   Scalar
```

- First time is total, and second time is “self” time.

Intel VTune and Advisor

- Available for download in the Intel OneAPI Package for free
- It is possible to run collection on Gadi and download locally for analysis in GUI
- First Lab will use these tools from the command line
- Notes on offline analysis to follow (useful for assignment)

Other Profiling Tools

- The profiling tools introduced above are not specific to C, they work with other languages such as Fortran
- We have concentrated on CPU performance, there are other useful tools
  - perf - powerful tool for measuring performance on Linux
  - vmstat, vm stat, memvis - virtual memory and CPU statics
  - mpstat, mpvis - parallel memory/CPU statics
  - netstat, nfsstat, nfsvis - network status and statistics
  - iostat, dkvis - I/O statics
  - sar - system activity report
  - top, prstat - list of most active processes
  - systat - system activity report
  - lockstat - kernel lock statics
  - dkstat - file status information

Benchmarks and Scientific Software Lifecycle

- When to stop optimizing?
- Comparing code to theoretical FLOPs or Memory Throughput

Benchmarking

Motivation

- Which machine is the fastest (for my applications)?
  - no simple answer: generally any single benchmark tests a single aspect of a machine
- To justify the purchase of a particular machine (eg NCI runs benchmarks)
- A problem set to compare two competing algorithms/methodologies

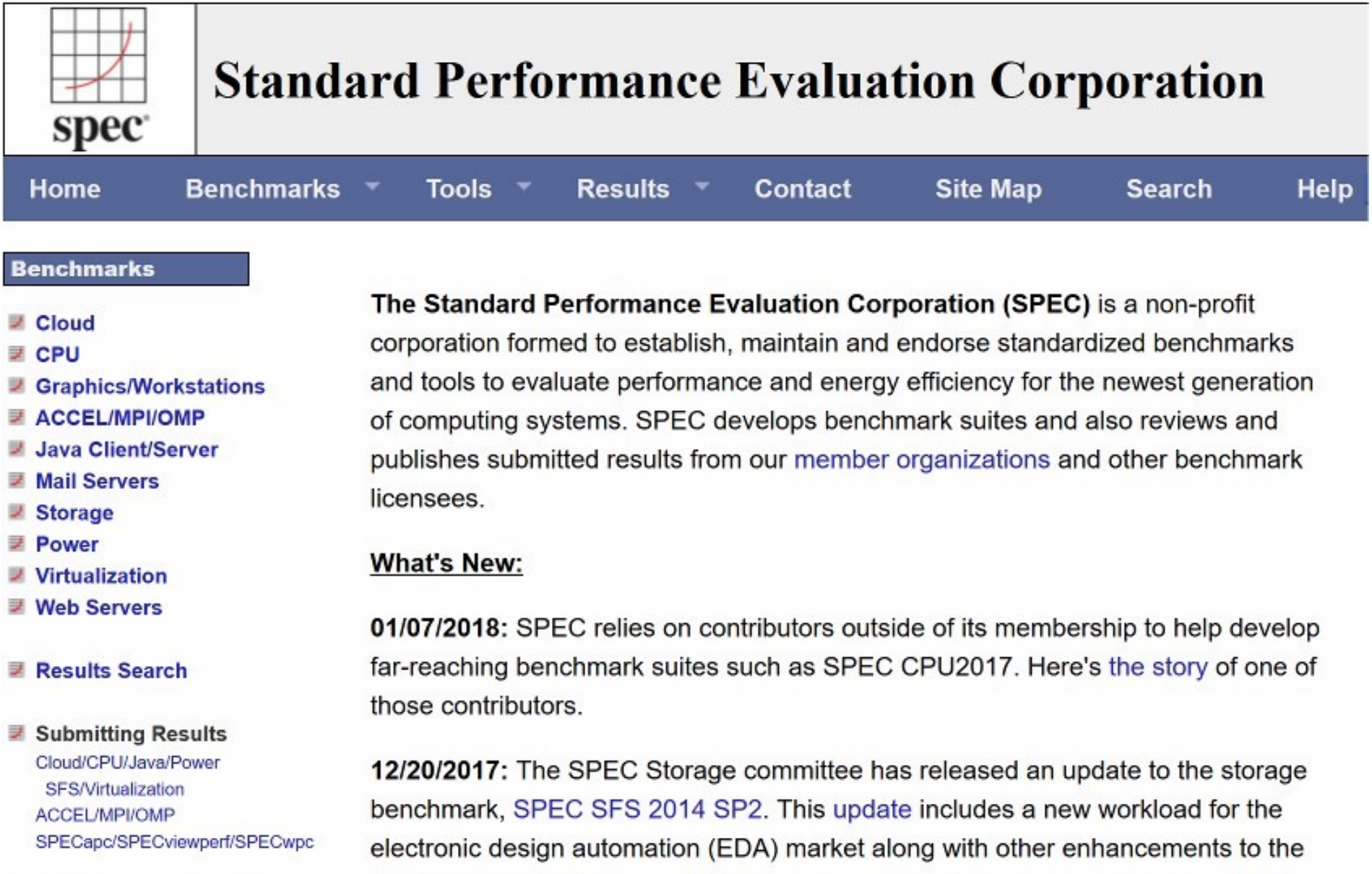
What benchmarks

- Use some third party benchmark
- Create your own benchmark representing your own application mix
  - typically has larger program / data sizes
  - generally, will expose new (or a different mix of) features of the algorithm-architecture interaction
  - exposes the effects of compilers

Standard Benchmarks

- Simple performance measures:
  - MIPS: Millions of Instr'ns Per Second
    - Requires a notion of a 'standard' instr'n set, eg. VAX MIPS
  - MFLOPS: Millions of Floating Point Operations Per Second
    - Peak: the speed that you are guaranteed never to exceed!
    - Sustained: the rate achieved by a "standard" computation, eg. LINPACK (http://www.top500.org/lists/linpack.html)
  - How meaningful are these?
- Benchmark suites: a set of small benchmark programs
  - eg. SPEC int/fp 2006 (http://www.spec.org)
  - TPC (database; http://www.tpc.org/),
  - STREAM (memory bandwidth; http://www.cs.virginia.edu/stream)
  - Gives some measure of "all-round" performance
  - Vendors play the benchmark game; users often disappointed

SPEC


spec
Standard Performance Evaluation Corporation
Home
Benchmarks
Tools
Results
Contact
Site Map
Search
Help
Benchmarks
Cloud
CPU
Graphics/Workstations
ACCEL/MPI/OMP
Java Client/Server
Mail Servers
Storage
Power
Virtualization
Web Servers
Results Search
Submitting Results
Cloud/CPU/Java/Power
SFS/Virtualization
ACCEL/MPI/OMP
SPECapc/SPECviewperf/SPECwpc
The Standard Performance Evaluation Corporation (SPEC) is a non-profit corporation formed to establish, maintain and endorse standardized benchmarks and tools to evaluate performance and energy efficiency for the newest generation of computing systems. SPEC develops benchmark suites and also reviews and publishes submitted results from our member organizations and other benchmark licensees.
What's New:
01/07/2018: SPEC relies on contributors outside of its membership to help develop far-reaching benchmark suites such as SPEC CPU2017. Here's the story of one of those contributors.
12/20/2017: The SPEC Storage committee has released an update to the storage benchmark, SPEC SFS 2014 SP2. This update includes a new workload for the electronic design automation (EDA) market along with other enhancements to the

The usefulness of Benchmarks

- The key thing with benchmarks is how they relate to your own algorithms
- For example, if you know your application is integer operation dominant then SPECint can be used to judge between different architectures
- If you algorithm uses a similar combination of operations as one of the STREAM benchmarks, you can use this as a guide as to how well your code is optimized.
- We will look more at benchmarks (particularly STREAM) and what they can show in labs and later lectures.